Ejercicio 1 "Pruebas de Caja Negra - Unitarias"

El objetivo es crear, a partir de las especificaciones recibidas de una clase, las pruebas unitarias que verifiquen el comportamiento requerido.

Objetivo.

Diseñar e implementar, mediante MSTest.Sdk, un conjunto de pruebas unitarias de caja negra que permita verificar el comportamiento especificado de las clases ServicioHash y ResultadoHash.

El diseño de las pruebas deberá basarse exclusivamente en las interfaces públicas y en los requisitos incluidos en este documento. No deberá suponerse ningún detalle interno de implementación que no figure expresamente en la especificación.

Durante esta práctica no se proporcionará la implementación de la clase. El diseño de las pruebas deberá realizarse exclusivamente a partir de las interfaces públicas y de los requisitos descritos en este documento.

REQUISITO FUNCIONAL

Las contraseñas no se almacenarán directamente. Para su almacenamiento seguro se generará un hash criptográfico utilizando una sal independiente para cada operación. Cada vez que se genere un hash a partir del mismo texto de entrada, deberán obtenerse una sal y un hash diferentes.

Diseño

Durante la fase de diseño se ha definido un componente denominado ServicioHash, cuya responsabilidad será gestionar la generación y validación de hashes criptográficos utilizados por la aplicación para el almacenamiento seguro de información sensible.

El componente deberá ofrecer mecanismos que permitan generar hashes a partir de un texto y verificar posteriormente su validez utilizando la información almacenada.

Especificaciones de "diseño de componente" para la clase ServicioHash

Interfaces públicas disponibles

Los estudiantes deberán diseñar e implementar las pruebas utilizando exclusivamente las interfaces públicas y los requisitos descritos en este documento. No deberá asumirse ningún detalle interno de implementación.

```
public class ServicioHash
{
        public ResultadoHash Generar(string texto);
 public bool Verificar(
                string texto,
                string hashAlmacenado,
                string salAlmacenada);
}

public class ResultadoHash
{
        public string Hash { get; init; }
        public string Sal { get; init; }
}
```

La implementación interna de estas clases no estará disponible durante la fase de diseño. Las pruebas deberán construirse únicamente a partir del comportamiento observable especificado en este documento.

Especificaciones de diseño del método Generar

Comportamiento esperado

1. Si el parámetro texto es null, una cadena vacía o contiene únicamente espacios en blanco, el método deberá lanzar una InvalidOperationException.
2. Si el texto es válido:
    a. Debe devolverse un objeto de tipo ResultadoHash.
    b. La propiedad Hash debe contener un valor no vacío.
    c. La propiedad Sal debe contener un valor no vacío.
3. Dos invocaciones consecutivas utilizando el mismo texto deben generar:
    a. Sales diferentes.
    b. Hashes diferentes.
4. El método nunca debe devolver valores nulos.

```
ResultadoHash Generar(string texto)
```

ED-01	**Disponibilidad del método Generar**	La clase ServicioHash deberá proporcionar un método público denominado Generar.
ED-02	**Parámetro de entrada**	El método Generar deberá recibir como parámetro un texto a partir del cual se generará un hash criptográfico.
ED-03	**Validación de entrada**	El método Generar deberá rechazar los valores nulos, vacíos o compuestos únicamente por espacios en blanco.
ED-04	**Gestión de entradas no válidas**	Cuando el método Generar reciba un valor nulo, vacío o compuesto únicamente por espacios en blanco, deberá lanzar una excepción de tipo InvalidOperationException.
ED-05	**Resultado de la operación**	Cuando la operación finalice correctamente, el método Generar deberá devolver una instancia de la clase ResultadoHash.
ED-06	**Generación de la sal**	Durante cada ejecución deberá generarse una sal independiente para utilizarla en el cálculo del hash.
ED-07	**Unicidad de los resultados**	Dos invocaciones independientes del método Generar realizadas con el mismo texto deberán producir sales diferentes.
ED-08	**Variabilidad de los hashes**	Como consecuencia del uso de sales diferentes, dos invocaciones realizadas con el mismo texto deberán producir hashes diferentes.
ED-09	**Integridad del resultado**	La instancia devuelta deberá contener valores válidos para todas sus propiedades.

Especificaciones del método Verificar

Comportamiento esperado

1) Debe devolver true cuando:

a) El texto coincide con el utilizado para generar el hash.
b) El hash almacenado es válido.
c) La sal almacenada es válida.

2) Debe devolver false cuando:
   a) El texto no coincide.
   b) El hash es incorrecto.
   c) La sal es incorrecta.
   d) Alguno de los parámetros es null.
   e) Alguno de los parámetros es una cadena vacía.
   f) Alguno de los parámetros contiene únicamente espacios.
3) Debe devolver false cuando los datos almacenados no tienen un formato válido.
4) El método nunca debe lanzar excepciones debido a datos almacenados incorrectos.Comportamiento esperado

```
bool Verificar(string texto,string hashAlmacenado,string salAlmacenada)
```

ED-10	**Disponibilidad del método**	La clase ServicioHash deberá proporcionar un método público denominado Verificar.
ED-11	**Parámetro de entrada**	El método deberá recibir: el texto a comprobar, el hash almacenado y la sal utilizada durante la generación del hash.
ED-12	**Verificación correcta**	El método deberá devolver true cuando el texto recibido corresponda al texto utilizado para generar el hash indicado y se proporcione la sal asociada.
ED-13	**Verificación incorrecta**	El método deberá devolver false cuando el texto recibido no corresponda al utilizado durante la generación del hash.
ED-14	**Validación de parámetros**	El método deberá devolver false cuando cualquiera de los parámetros recibidos sea nulo, vacío o contenga únicamente espacios en blanco.
ED-15	**Gestión de datos inválidos**	El método deberá gestionar de forma controlada los datos de entrada con formato incorrecto.
ED-16	**Comportamiento ante formatos incorrectos**	Si el hash o la sal almacenados presentan un formato no válido, el método deberá devolver false.
ED-17	**Robustez**	El método no deberá propagar excepciones provocadas por datos almacenados incorrectos.

Especificaciones de “diseño de componente” para la clase ServicioHash

```
public class ResultadoHash {
	public string Hash { get; init; }
	public string Sal { get; init; }
}
```

ED-18	**Especificaciones del Almacenamiento del hash**	La clase ResultadoHash deberá disponer de una propiedad denominada Hash destinada a contener el hash generado.
ED-19	**Almacenamiento de la sal**	La clase ResultadoHash deberá disponer de una propiedad denominada Sal destinada a contener la sal utilizada durante el proceso de generación.
ED-20	**Consistencia del resultado**	Toda instancia de ResultadoHash devuelta por el componente deberá contener valores no nulos y no vacíos para las propiedades Hash y Sal..

ID	REQUISITO	NOMBRE DE LA PRUEBA	ENTRADA	RESULTADO ESPERADO
CP-01				
CP-02				
CP-03				
CP-04				
CP-05				
CP-06				
CP-07				
CP-08				
CP-09				
CP-10				

La tabla puede ampliarse con todas las filas necesarias. Las pruebas parametrizadas podrán representarse mediante un único caso de prueba, siempre que se indiquen claramente todos los conjuntos de datos utilizados.